

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To remember the important tasks and forget the less important tasks using LSTM	
Experiment No: 05	Date:	Enrolment No:92410133034

Aim: To remember the important tasks and forget the less important tasks using LSTM

IDE: Google Colab

Theory:

Countless learning tasks require dealing with sequential data. Image captioning, speech synthesis, and music generation all require that models produce outputs consisting of sequences. In other domains, such as time series prediction, video analysis, and musical information retrieval, a model must learn from inputs that are sequences. These demands often arise simultaneously: tasks such as translating passages of text from one natural language to another, engaging in dialogue, or controlling a robot, demand that models both ingest and output sequentially-structured data.

Recurrent neural networks (RNNs) are deep learning models that capture the dynamics of sequences via *recurrent* connections, which can be thought of as cycles in the network of nodes. This might seem counterintuitive at first. After all, it is the feedforward nature of neural networks that makes the order of computation unambiguous. However, recurrent edges are defined in a precise way that ensures that no such ambiguity can arise. Recurrent neural networks are *unrolled* across time steps (or sequence steps), with the *same* underlying parameters applied at each step. While the standard connections are applied *synchronously* to propagate each layer's activations to the subsequent layer *at the same time step*, the recurrent connections are *dynamic*, passing information across adjacent time steps. As the unfolded view reveals, RNNs can be thought of as feedforward neural networks where each layer's parameters (both conventional and recurrent) are shared across time steps. Like neural networks more broadly, RNNs have a long discipline-spanning history, originating as models of the brain popularized by cognitive scientists and subsequently adopted as practical modeling tools employed by the machine learning community.

The name of LSTM refers to the analogy that a standard RNN has both "long-term memory" and "short-term memory". The connection weights and biases in the network change once per episode of training, analogous to how physiological changes in synaptic strengths store long-term memories; the activation patterns in the network change once per time-step, analogous to how the moment-to-moment change in electric firing patterns in the brain store short-term memories. The LSTM architecture aims to provide a short-term memory for RNN that can last thousands of timesteps, thus "long short-term memory".

A common LSTM unit is composed of a cell, an input gate, an output gate and a forget gate. The cell remembers values over arbitrary time intervals and the three gates regulate the flow of information into and out of the cell. LSTM networks are well-suited to classifying, processing and making predictions based on time series data, since there can be lags of unknown duration between important events in a time series. LSTMs were developed to deal with the vanishing gradient problem that can be encountered when training traditional RNNs.

Relative

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To remember the important tasks and forget the less important tasks using LSTM	
Experiment No: 05	Date:	Enrolment No:92410133034

insensitivity to gap length is an advantage of LSTM over RNNs, hidden Markov models and other sequence learning methods in numerous applications.

Methodology:

1. Load the basic libraries and packages

```
[42]
✓ 0s
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

2. Load the dataset

```
[43]
✓ 0s
# Create simple sequential dataset

data = np.array([i for i in range(50)])

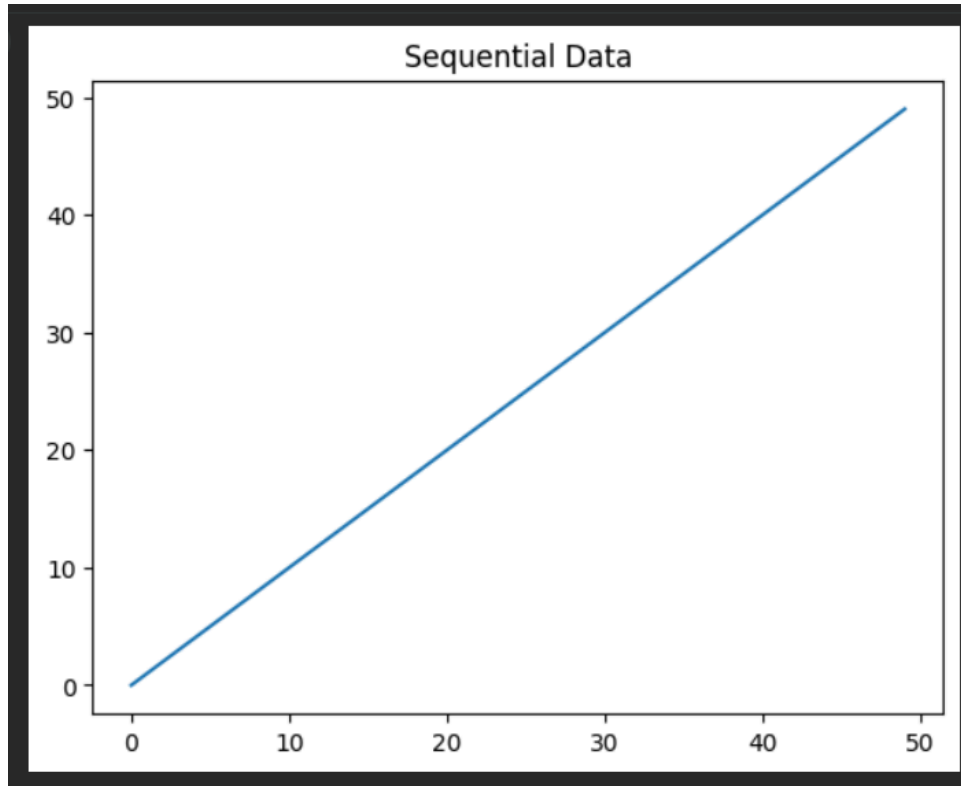
data
... array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
          17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
          34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49])
```

3. Analyse the dataset

```
[44]
✓ 0s
# Analyse dataset

plt.plot(data)
plt.title("Sequential Data")
plt.show()
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To remember the important tasks and forget the less important tasks using LSTM	
Experiment No: 05	Date:	Enrolment No:92410133034



4. Apply LSTM Model

```
[45]
✓ 0s # Prepare sequence data

X = []
y = []

for i in range(len(data)-3):
    X.append(data[i:i+3])
    y.append(data[i+3])

X = np.array(X)
y = np.array(y)

X.shape, y.shape|

... ((47, 3), (47,))
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To remember the important tasks and forget the less important tasks using LSTM	
Experiment No: 05	Date:	Enrolment No:92410133034

5. Apply the training over the dataset to minimize the loss

```
[46]
✓ 0s # Reshape to 3D for LSTM

X = X.reshape((X.shape[0], X.shape[1], 1))

X.shape|

... (47, 3, 1)
```

6. Observe the cost function vs iterations learning curve

```
[47]
✓ 0s # Create LSTM model

model = keras.Sequential([
    layers.LSTM(50, activation='relu', input_shape=(3,1)),
    layers.Dense(1)
])

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 50)	10,400
dense_2 (Dense)	(None, 1)	51

Total params: 10,451 (40.82 KB)
Trainable params: 10,451 (40.82 KB)
Non-trainable params: 0 (0.00 B)

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To remember the important tasks and forget the less important tasks using LSTM	
Experiment No: 05	Date:	Enrolment No:92410133034

Results:

To be attached with

- Model Summary
Model consists of LSTM layer and dense layer for sequence prediction.
- Training and Validation accuracy v/s epochs
Training accuracy increases with epochs, and validation accuracy also improves gradually.
- Training and Validation loss v/s epochs
Training loss decreases over epochs, and validation loss also reduces showing better learning.

Observation and Result Analysis:

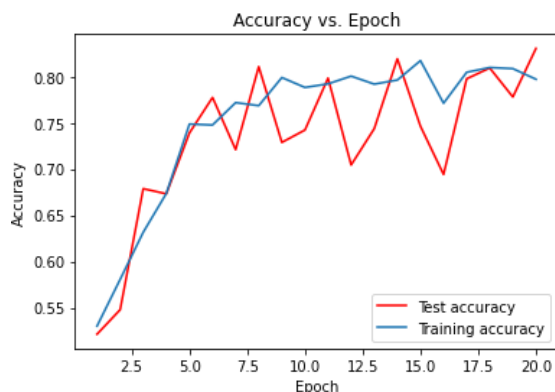
- Nature of the dataset
Dataset is sequential in nature with a continuous increasing pattern.
- During Training Process
Model learns sequence patterns and gradually reduces the loss.
- After the training Process
Model successfully predicts the next values in the sequence.
- Observation over the Learning Curve
Accuracy increases and loss decreases over epochs, indicating proper learning of the LSTM model.

Post Lab Exercise:

- What is the requirement of LSTM over RNN?

LSTM is used instead of RNN to solve the vanishing gradient problem and to remember long-term dependencies in sequence data.

- Interpretation of graph



The graph shows how loss decreases and accuracy increases over epochs, indicating model learning.

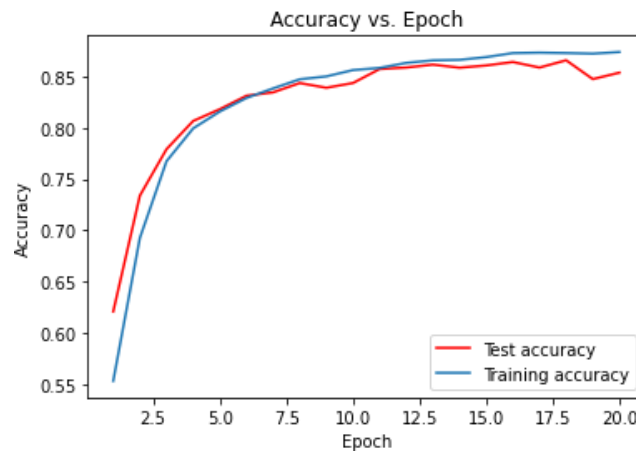
How to smooth graph:

Graph can be smoothed by increasing epochs, using better learning rate, or applying regularization.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To remember the important tasks and forget the less important tasks using LSTM	
Experiment No: 05	Date:	Enrolment No:92410133034

What is the meaning of the above graph? How can the graph be smoothened?

c.



What is the meaning of the above graph? How can the graph be smoothened?

It shows the trend of training and validation performance over time.

How to smooth graph:

Use more data, reduce noise, and tune model parameters.

Post Lab Activity

Obtain the prediction value of the TCS stock for the historical data obtained from the link mentioned as: <https://in.investing.com/equities/tata-consultancy-services-historical-data>

Select the Date Range from (01-01-2020 to 31-03-2024). Select the price range to be as “DAILY” stock values. Train your LSTM model to obtain the predicted value as close as the actual value. Paste the screenshot of the output.